Project - High Level Design

Autonomous Legal Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Jogendar Das Bairagi	EN22ME304045
2.	Roshan Bankar	EN22EL301072
3.	Vivek Gangrade	EN22CS3011098
4.	Maahi Maheshwari	EN22ME304057
5.	Shubhangi Prajapat	EN22ME304094

Group Name : 04D10

Project Number : DO-17

Industry Mentor Name : Mr. Ashwin Krishnan

University Mentor Name : Prof Maya Yadav Baniya

Academic Year : 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. **Data Design**
   3.1 Data Model
   3.2 Data Access Mechanism
   3.3 Data Retention Policies
   3.4 Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The Autonomous Legal Researcher Agent is an AI-driven system designed to assist users in retrieving, analyzing, and summarizing legal information. It leverages Large Language Models (LLMs) combined with real-time web search capabilities to produce structured and reliable legal reports.

The system ensures domain-specific accuracy by restricting queries to legal topics and presenting outputs in a standardized, human-readable format.

1.1. Scope of the Document

This document covers:

- Detailed system architecture and design
- Component-level interactions
- Data and control flow
- API integrations
- Design decisions and trade-offs

It focuses strictly on **low-level implementation details**.

1.2. Intended Audience

- AI/ML Engineers
- Backend & Frontend Developers
- System Designers
- Academic Evaluators

1.3. System Overview

The system acts as a **domain-specific intelligent agent** that:

- Accepts user queries via UI
- Filters non-legal queries
- Retrieves real-time legal data
- Generates structured legal explanations
- Produces downloadable reports

Core Capabilities:

- Context-aware response generation
- Multi-source data aggregation
- Structured output formatting
- Lightweight UI interaction

Architecture Flow

User → UI (Gradio)
↓
Controller (process function)
↓
Validation Layer
↓
Agent Layer (legal_agent)
↓
Search Layer (Tavily API)
↓
Context Builder
↓
LLM Layer (Groq API)
↓
Formatter
↓
PDF Generator
↓
Response to UI

2. System Design

The system design defines the overall structure and organization of the Autonomous Legal Researcher Agent. It follows a modular and layered architecture to ensure clear separation of concerns, efficient data processing, and seamless integration with external services such as search APIs and LLMs. The design focuses on scalability, maintainability, and reliable handling of user queries.

2.1. Application Design

1. UI Module

- Built using Gradio
- Handles user input and output rendering
- Provides examples and file download

2. Controller Module

- Function: process()
- Responsibilities:
  - Input validation
  - Flow control
  - Error handling

3. Validation Module

- Function: is_legal_keyword()
- Ensures domain restriction
- Prevents irrelevant processing

4. Agent Module

- Function: legal_agent()
- Core logic of system
- Coordinates search + LLM

5. Search Module

- Uses Tavily API
- Fetches real-time data
- Returns structured search results

6. LLM Module

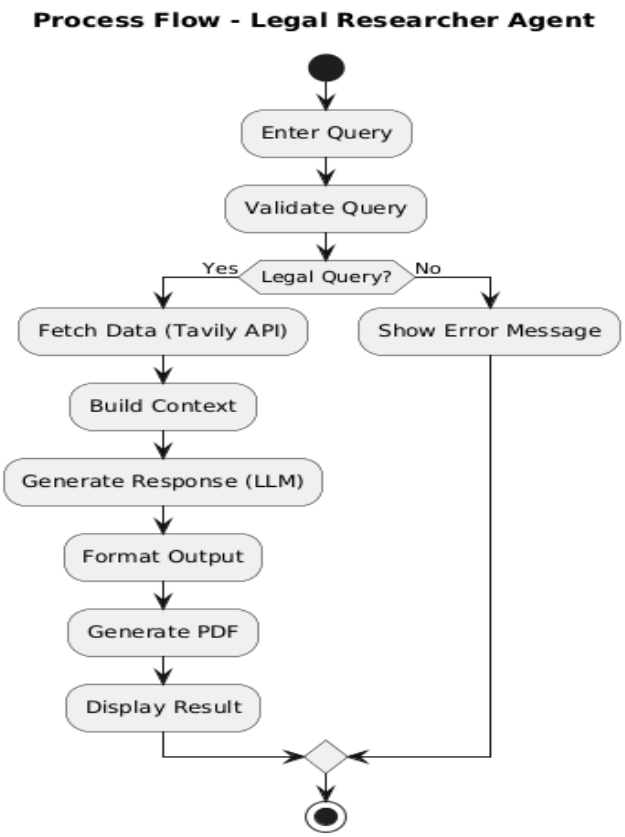
- Uses Groq API
- Generates structured responses
- Handles reasoning and summarization

7. PDF Module

- Uses FPDF
- Converts structured text into formatted document

2.2 Process Flow

The process begins when the user submits a query through the interface, which is handled by the controller. The system validates whether the query is legal in nature. If invalid, an error message is returned. If valid, the legal agent retrieves relevant data using the Tavily API, constructs context, and sends it to the LLM for response generation. The generated response is then formatted and converted into a PDF report. Finally, both the structured output and downloadable PDF are displayed to the user.



The system follows a structured information flow where data is progressively transformed from raw user input into a formatted legal report.

2.3. Information Flow

The system follows a structured information flow where data is progressively transformed from raw user input into a formatted legal report.

1. User Input

- **Input:** Raw text query from user
- Example: *"What are cybercrime laws in India?"*

2. Validation Layer

- **Input:** User query
- **Output:** Validation result (LEGAL / INVALID)
- Purpose: Ensure only legal queries proceed further

3. Search Layer (Tavily API)

- **Input:** Validated query
- **Output:**
  - URLs
  - Content snippets
- Purpose: Retrieve real-time relevant data

4. Context Builder

- **Input:** Search results
- **Output:** Combined context text
- Purpose: Convert multiple sources into a single input for LLM

5. LLM Processing Layer

- **Input:** Prompt + Context
- **Output:** Structured legal response

Example output:

- Clear Explanation
- Key Laws
- Important Points
- Conclusion

6. Formatting Layer

- **Input:** Raw LLM response
- **Output:** Cleaned and structured text

- Purpose:
  - Remove unwanted tokens (**, emojis)
  - Standardize headings

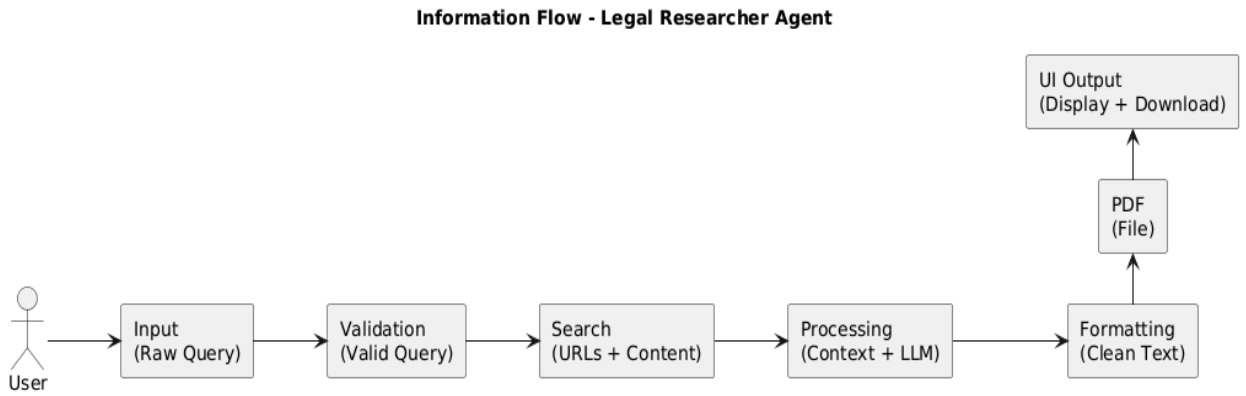
7. PDF Generation

- **Input:** Formatted text + Query
- **Output:** PDF file
- Purpose: Provide downloadable report

8. Output to UI

- **Input:** Final report + PDF file
- **Output:**
  - Displayed text
  - Download option

Information Flow Diagram



2.4. Component Design

1. legal_agent()

Purpose:
Core component responsible for retrieving data, building context, and generating structured legal responses using LLM.

Input:

- User query (string)

Output:

- Structured legal report (text)

Internal Workflow:

- Sends query to Tavily API for web search
- Receives search results (URLs + snippets)
- Extracts and aggregates relevant content
- Builds a unified context string
- Constructs a structured prompt using query + context
- Sends prompt to LLM (Groq API)
- Receives structured response
- Returns formatted report

Dependencies:

- Tavily API
- Groq LLM API

2. Generate_pdf()

Purpose:
Converts structured text output into a formatted and downloadable PDF report.

Input:

- Generated report (text)
- User query (string)

Output:

- PDF file path

Internal Workflow:

- Initializes PDF document
- Adds title using user query
- Parses report into lines
- Detects sections (Explanation, Laws, Conclusion)
- Applies formatting: Headings, Bullet points, Numbered list.
- Cleans unsupported characters
- Writes content into PDF
- Saves file and returns path

Dependencies:

- FPDF library

3. process()

Purpose:
Acts as the central controller that manages the complete execution flow of the system.

Input:

- User query (string)

Output:

- Structured report
- PDF file

Internal Workflow:

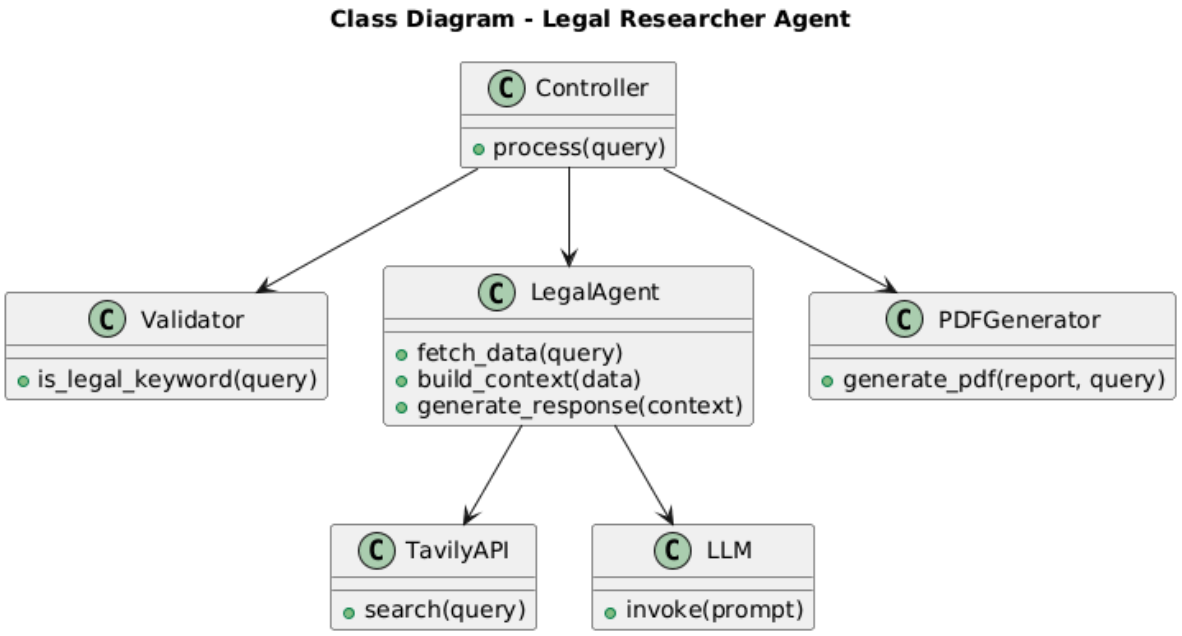
- Receives query from UI
- Validates query using is_legal_keyword()
- Decision handling:
    - Invalid → return error message
    - Valid → continue processing
- Calls legal_agent() to generate report
- Calls generate_pdf() to create PDF
- Returns both report and PDF to UI

Responsibilities:

- Routing between components
- Validation handling
- Error handling (try-catch)
- Output aggregation

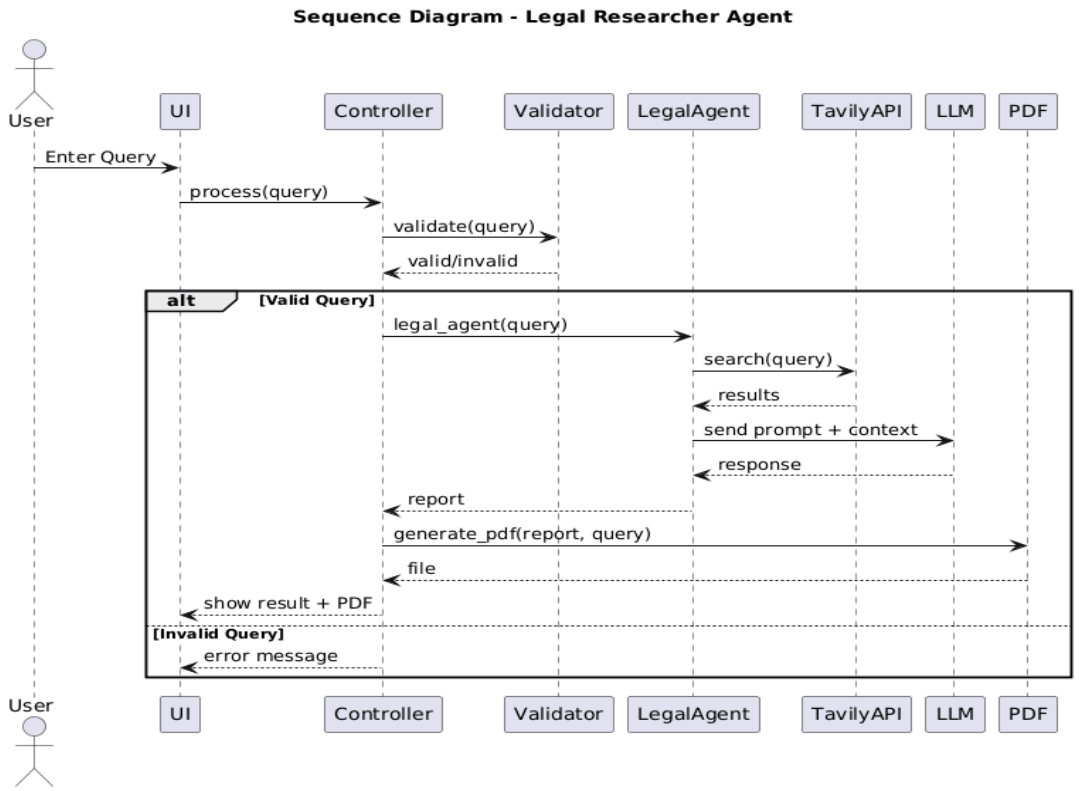
Class Diagram

The class diagram represents the static structure of the system, showing key components such as Controller, LegalAgent, Validator, and PDFGenerator, along with their relationships and responsibilities



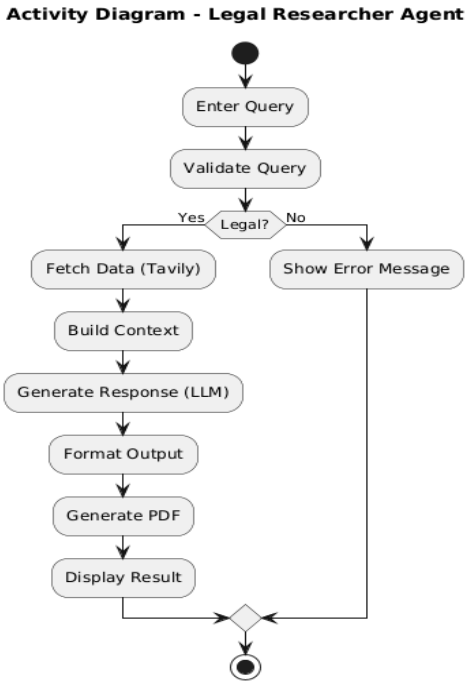
Sequence Diagram

The sequence diagram illustrates the interaction between system components over time. It shows how the user query flows through validation, processing, API calls, and finally returns the response and PDF output.



Activity Diagram

The activity diagram represents the step-by-step workflow of the system, including decision points such as query validation and the sequence of operations leading to output generation.



2.5. Key Design Considerations

Accuracy vs Speed

- Groq chosen for fast inference
- Tavily ensures factual grounding

Scalability

- Modular components allow future expansion (multi-domain AI)

Reliability

- Keyword-based validation avoids incorrect outputs

Maintainability

- Clear separation of modules

Error Handling

- Try-catch in controller
- Input validation

Security

- API keys handled via environment variables

2.6. API Catalogue

Tavily API

- Method: Search
- Input: Query
- Output: JSON results

Groq API

- Method: LLM inference
- Input: Prompt
- Output: Text response

Internal APIs

Function	Purpose
process()	Controller
legal_agent()	Core logic
generate_pdf()	Report generation

3. Data Design

3.1 Data Model

The system follows a dynamic, unstructured data model as it primarily processes real-time textual data rather than storing structured datasets.

Data Entities:

- User Query
  - Type: String
  - Description: Input provided by user
- Search Results
  - Type: JSON (from Tavily API)
  - Fields: URLs, content snippets
- Context Data
  - Type: String
  - Description: Aggregated content from multiple sources
- LLM Response
  - Type: String
  - Description: Structured legal report
- PDF Report
  - Type: File (PDF)
  - Description: Final formatted

3.2 Data Access Mechanism

The system uses API-based data access mechanisms:

- Tavily API → Fetches real-time web data
- Groq API → Processes and generates responses

Flow:

- Query → API call → JSON response → processing

No database is used; all data is processed in-memory.

3.3 Data Retention Policies

The system follows a stateless and temporary data retention model:

- User queries are not permanently stored
- API responses are used only during execution
- Generated PDF files are stored temporarily in runtime
- PDF can be downloaded for permanent storage on your local drive.

Benefits:

- Improved privacy
- Reduced storage overhead
- No long-term data dependency

3.4 Data Migration

Currently, data migration is not applicable as:

- No persistent database is used
- Data is generated and processed in real-time

Future Scope:

- Integration with database (MongoDB, Firebase)
- Knowledge base storage for reuse

4. Interfaces

The system interacts with both internal and external interfaces:

User Interface

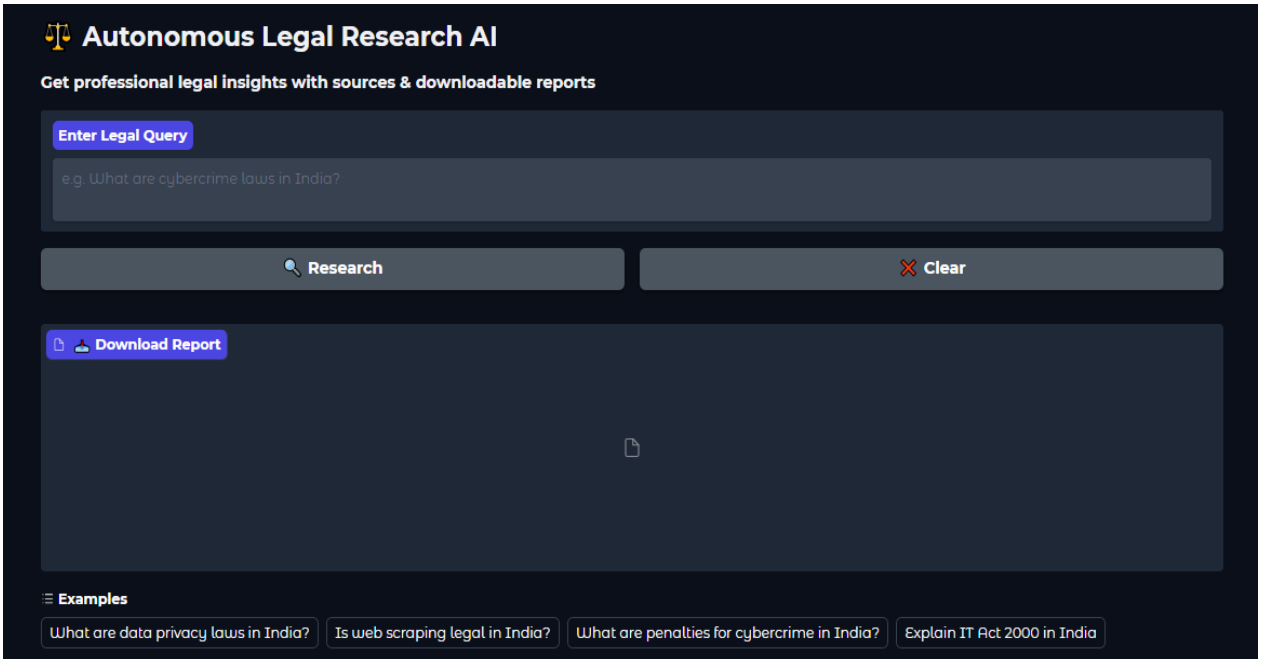
- Built using Gradio
- Accepts input queries
- Displays structured output and PDF

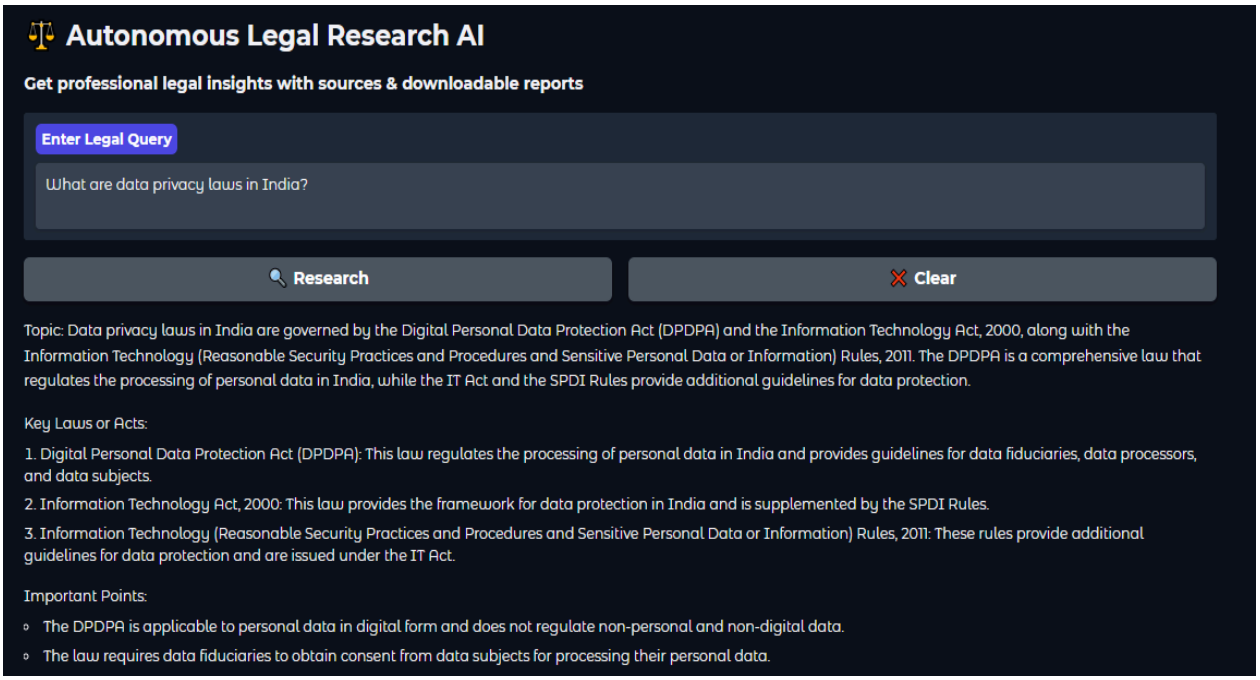
External APIs

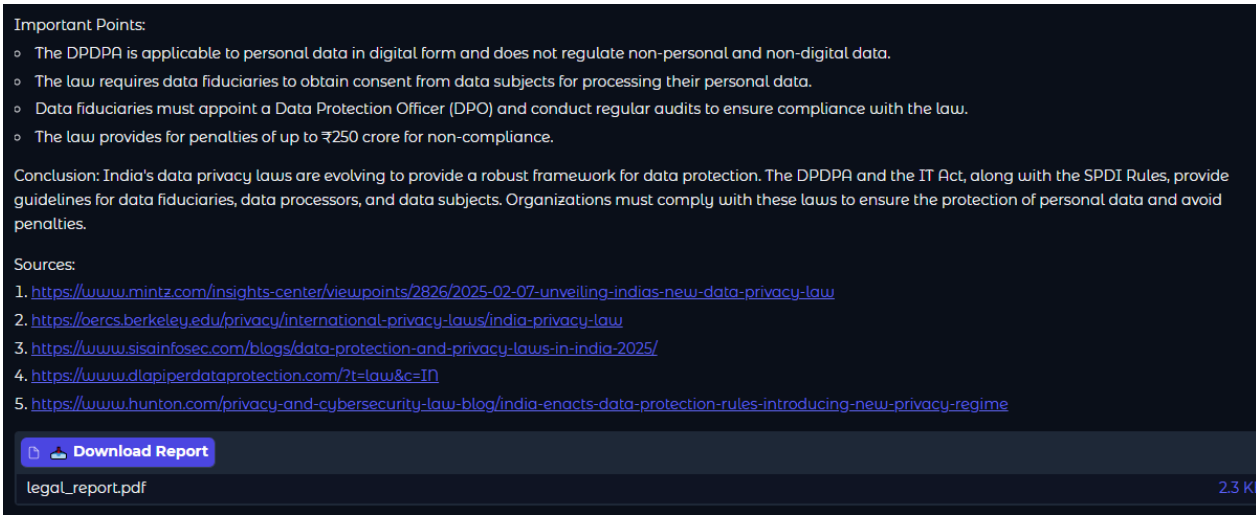
- Tavily API → Web search
- Groq API → LLM processing

Internal Interfaces

- Function calls:
  - process()
  - legal_agent()
  - generate_pdf()







5. State and Session Management

The system follows a stateless architecture:

- Each request is processed independently
- No session data is maintained between requests

Characteristics:

- No user session tracking
- No login/authentication layer
- Each query → fresh execution cycle

6. Caching

Currently, no caching mechanism is implemented.
Reason:

- Data is fetched in real-time
- Responses depend on latest search results

Future Enhancement:

- Cache frequent queries
- Store API responses temporarily
- Reduce API calls and improve performance

7. Non-Functional Requirements

7.1 Security Aspects

- API keys are stored securely (environment variables)
- No sensitive user data is stored
- Input validation prevents misuse
- External API communication over HTTPS

Risks Mitigated:

- Unauthorized API access
- Data leakage
- Invalid input exploitation

7.2 Performance Aspects

- Fast LLM inference using Groq API
- Lightweight UI (Gradio)
- Minimal processing overhead (no DB)

Performance Factors:

- API response time

- Network latency
- LLM processing speed

Optimization Scope:

- Implement caching
- Reduce prompt size
- Optimize context building

8. References

- Tavily API Docs – https://docs.tavily.com
- Groq API Docs – https://console.groq.com/docs
- FPDF Docs – https://pyfpdf.readthedocs.io
- Gradio Docs – https://www.gradio.app/docs
- LangChain Docs – https://docs.langchain.com